基于 Zynq UltraScale+ XCZU3EG 的实时渲染、风格化与超分融合系统

ARM 几何前端 × FPGA 固定功能像素后端 × 行流式空间超分

摘 要

本作品面向资源受限的嵌入式实时图形场景，在单颗 Zynq UltraScale+ XCZU3EG 上构建了一条「低分辨率渲染 → 片内风格化 → 空间超分 → 高分辨率显示」的完整视觉增强链路。系统把图形负载切分为两层：ARM Cortex-A53 承担游戏逻辑与全部几何运算（顶点解包、矩阵变换、光照、视锥与用户裁剪、图元装配、边函数与属性梯度准备、三分片调度），可编程逻辑承担固定功能像素后端（边漫游光栅化、纹理单元、深度与混合、条带帧缓冲），两者之间只传递一条 192 字节的整数化三角形描述符命令流。这一划分使 FPGA 侧无需重建完整几何管线。基线 v23 为 64,901 LUT / 325 DSP / 216 BRAM；频率与资源协同优化后的上板最高配置 PL250 为 63,349 LUT / 179 BRAM / 346 DSP，GPU 域实跑 **202.752 MHz**（按 203.004 签核，WNS +0.224 ns），CNN 域 **250 MHz**（按 254.001 签核，WNS +0.116 ns），超分保持独立 200 MHz 域，三个域零失败端点。

在像素后端之上，作品实现了完整空间超分核（EASU 12 抽头边缘自适应重采样 + RCAS 5 抽头对比度自适应锐化），把 400×240 的 RGB565 源画面重建为 800×480 并直接扫描输出到 RGB LCD。超分工作频率 200 MHz，与 100 MHz 几何/控制域、33.333 MHz 像素域构成三时钟结构，通过 bundled-data 翻转握手实现跨域整帧一致性发布。为缩短「计算完成→显示可用」的间隙，把行缓存按扫描分组组织，并让全部缩放模式复用同一块读突发缓存，避免下游反压沿 DDR 读通道回传到超分计算级。在 400×240→800×480、64^3 世界、视距 32 的 120 秒长测中，新输出帧率达 57.089 帧/秒；视距 16 时为 60.106 帧/秒。同场景 20 秒对照中，相同预取程序的新输出由早期版本的 36.83 提升到 47.83 帧/秒。

作品进一步接入了三层 INT8 卷积神经风格迁移。为在小容量器件上腾出存储空间，我们把

原本「双整帧特征库交替读写」改为「单整帧库 + 两行循环暂存 + 原地写回」：利用 3×3 卷积行扫描的依赖窗口，在旧输入行最后一批读者退出后再覆盖它，逻辑特征存储由 245,760 字节降至 125,440 字节，核心 BRAM 由 55 块降至 28.5 块，整机由 216 块降至 189.5 块，而卷积、边界复制、舍入、ReLU 与饱和运算逐值不变。93 个用例、279 层输出与参考实现零差异另设「提前覆盖」负对照确认验证器有效。该候选已通过整机布线、配置审计与实机 A/B/A 回归 CNN 推理耗时随接口与频率演进：原子 12.873 ms → 分组 9.321 ms（−27.6%）→ 分组 + 250 MHz 共 1,430,784 周期 = **5.723 ms**；三个数字对应不同位流，不能混用。

系统最终以统一图形菜单承载十五款游戏：八款原有 3D 游戏，加上 rRootage 与六款自研原生规则小游戏。菜单支持分页、搜索、收藏、最近游玩、逐游戏私有库与位流 profile、全局返回热键与逐游戏超分开关。十五款在 PL250 上完成统一复测，CNN + 超分串联输出 45.0–55.8 帧/秒；arcade-snake 通过 120 秒长测，6,692 帧 / 55.76 帧/秒。

关键词：Zynq UltraScale+；固定功能图形管线；空间超分；INT8 卷积；片上存储复用

第一部分　作品概述

1.1　功能与特性

作品是一台可独立运行、可用实体键盘鼠标操作的多游戏实时图形平台，核心能力分为四层。

- 实时 3D 渲染：ARM 侧以固定功能 OpenGL 语义完成几何处理，PL 侧完成光栅化、双线性纹理采样、16 位深度测试与颜色混合，输出 400×240 RGB565 画面。
- 空间超分增强：把 400×240 重建到 800×480，含边缘自适应重采样与对比度自适应锐化两级算子，并提供最近邻、双线性、超分三种放大路径供同场景对照。
- 神经风格迁移：三层 INT8 卷积网络在 PL 内完成推理，支持动漫与油画两套权重，可切换强度，并保留原图亮度细节融合与 HUD 保护区域。
- **已实现「PL CNN → ARM 400×240 细节融合 → PL FSR1 → 800×480 LCD」的串联链路**：融合在 ARM、卷积与超分在 PL，**不新增 FPGA 资源。**

- 多游戏统一入口：中文菜单已扩到 **十五款** 游戏（八款原有 + 七款新增），支持分页、关键词搜索、收藏与最近游玩、分类筛选、键鼠导航、显示偏好保存，并有 Ctrl+Alt+M 全局返回；切换时按「正常关闭→排空→恢复位流→重建菜单」的顺序执行。

此外，系统对外提供完整的可观测性：GPU 侧暴露 16 字段只读状态（帧号、换页、拷贝、呈现确认、五类错误计数、命令拍数、忙周期），主机侧可通过串口读取并与配对单调时钟对齐，因此每一项性能数字都能追溯到具体的完成帧与时间窗口。

1.2 应用领域

本作品的目标场景是「算力与存储都受限、但需要真实 3D 画面与可感知画质提升」的嵌入式显示终端。

- 工业 HMI 与手持仪表的 3D 可视化：器件容量通常在数万 LUT 量级，无法容纳完整几何硬件，本作品的 ARM 几何 + PL 像素划分正对应这类约束。
- 车载与船载信息屏：需要在固定刷新节拍下稳定输出，且对撕裂、错帧零容忍；本作品的四层完成屏障（AXI 写排空、换页栅栏、帧桥完成码、消隐期发布）为此设计。
- 教育与科普演示：低分辨率渲染加超分是讲解「算法质量与硬件成本取舍」的直观载体，作品内置最近邻/双线性/超分三路同源对照，可直接用于教学。
- 复古游戏与模拟器前端：十五款游戏共用同一套硬件后端与菜单，其中六款经典小游戏是项目自研的原生规则实现（`ports/arcade/rules.hpp`），说明该架构可作为「一类应用」而非「一款游戏」的承载平台。

与通用 GPU 方案相比，本作品不追求绝对吞吐，而追求在极小器件上把「渲染—增强—显示」做成一条可验证、可回退、可解释的完整链路，因此更适合作为嵌入式图形教学与工程验证平台。

1.3 主要技术特点

一、几何与像素的分层切分。系统把 12 个几何阶段全部留在 ARM，PL 只接收「已经算好边函数、包围盒、w 梯度、颜色梯度、深度梯度与纹理坐标梯度的定点参数包」。这一划分避免

了在小器件上重建完整几何流水线——我们曾实测，把完整几何核搬进 PL 需要 91,645 LUT，是剩余预算的 2.82 倍。

二、能力按 profile 声明，而不是笼统声称。本作品共有四个已布线的硬件配置，能力互不相同，因此所有能力描述都按配置给出（见 3.2.2 表）：默认 v23 为 **单 TMU / 128×128 纹理 / 无模板缓冲**；Java 主线为 **双 TMU / 256×256 纹理 / 无模板缓冲 / CNN 为 stub**；PL250 上板配置为 **单 TMU / 128×128 / 无模板 / 真实 CNN / FSR 200 MHz。本作品不启用模板缓冲**，相关效果在游戏侧被关闭，这是已知且已声明的取舍。

三、三分片条带渲染。400×240 共 96,000 像素，超过片内 32,768 像素的条带缓冲，因此按 3 条带 × 80 行调度，逐带做 Y 方向重定基并把带偏移写进硬件寄存器，使 PL 侧完全不必知道「当前是第几片」。

四、行流式超分与突发缓存复用。超分不做整帧中间态往返，而是按行窗口计算；全部缩放模式复用同一块读突发缓存，使下游停顿不能占住 DDR 读响应通道。

五、存储复用而非算术削减。CNN 的 BRAM 回收通过改变存储调度实现，而非降低通道数或层数，因此画质无损、权重不变、参考输出逐值一致。

1.4 主要性能指标

指标	数值	测试条件
渲染分辨率 / 显示分辨率	400×240 → 800×480	RGB565 源，XRGB8888 显示缓冲
LCD 扫描节拍	60.125 Hz	33.3333 MHz ÷ (1056 × 525)
超分长测帧率（视距 32）	57.089 帧/秒	120 秒，64^3 世界，源 57.107
超分长测帧率（视距 16）	60.106 帧/秒	120 秒，源 60.132
同场景 20 秒对照	36.83 → 47.83 帧/秒	早期版本 → 当前版本
GPU 时钟域（上板最高）	**202.752 MHz**（签核 203.004 MHz）	WNS +0.224 ns
CNN 时钟域（上板最高）	**250 MHz**（签核 254.001 MHz）	WNS +0.116 ns
FSR 时钟域	200 MHz（独立 MMCM）	与 GPU 域异步
整机资源（v23 基线）	64,901 LUT / 71,349 FF / 325 DSP	容量 70,560 / 141,120 / 360
整机资源（PL250 上板）	**63,349 LUT / 68,204 FF / 179 BRAM / 346 DSP**	90.5% / 48.3% / 82.9% / 96.1%
BRAM 占用（v23）	216 / 216（100%）	CNN 回收后降至 189.5 / 216
布线时序（v23）	WNS +0.306 ns / WHS +0.010 ns	零失败端点
CNN 推理耗时（三档）	12.873 / 9.321 / **5.723** ms	原子 100M / 分组 100M / 分组 250M
CNN 核心周期	462,816（原地版）→ **1,430,784**（稀疏 250M）	周期增加但时钟提高 2.5×
游戏库规模	15 款（8 原有 + 7 新增）	七款新增均过实机 PL FSR
源呈现提速（Neverball）	28.97 → 45.17 FPS（+55.9%）	实际开局，async 呈现
源呈现提速（Quake III）	30.27 → 54.73 FPS（+80.8%）	com_maxfps 30→60
CNN + FSR 串联吞吐	约 44–50 FPS	七游戏短测，非游戏完成帧
CNN + FSR 提前通知增益	46.68 → 50.70 FPS（+8.62%）	Neverputt 同球洞 ABCCBA
PL250 十五游戏输出帧率	**45.0 – 55.8 FPS**	CNN+FSR 串联，8 s 短测
PL250 120 秒长测	**6,692 帧 / 55.76 FPS**	arcade-snake，fsr_wait 0.002 ms

上述数字均为已完成的整机布线后实测或板上计数，非 HLS 估算。其中帧率统计口径为「硬件呈现确认增量 ÷ 配对单调时钟」，与 LCD 扫描频率严格区分。

1.5 主要创新点

- 命令级可验证的切分：分界物是 192 字节整数描述符而非顶点流，PL 侧不含任何几何运算

- 行流式超分：以行窗口与突发缓存复用替代整帧中间态，不增加 DDR 往返。
- CNN 存储原地复用：单整帧 + 两行暂存替换双整帧库，回收 26.5 块 BRAM 且逐值一致。
- 四层帧所有权保护链：任一层缺失都会出现「撕裂但计数正常」的假通过。
- 可回退的多 profile 运行：逐游戏独立库与位流，失败即退回并保留隔离记录。

1.6 设计流程

作品按「先建链路、再找瓶颈、后做取舍」的顺序推进，共四个阶段。

- 链路打通：修复启动链（U-Boot 控制设备树与旧树阶段标记）、DMA 缓冲生命期与 LCD 布局，使未修改的商业游戏首次在板上稳定出图。
- 能力扩展：把像素后端从 128×128 基线推进到可承载现代游戏；Java 主线另有双 TMU + 256×256 配置（65,624 LUT），但它与 CNN 不能同存，**默认位流仍为单 TMU + 128×128。**
- 增强叠加：把超分从 ARM 参考实现逐步迁移为完整 FPGA 实现，依次解决数值语义、行缓存分组、读突发缓存三个瓶颈，每步都以黄金图与 LCD 抽样验证。
- 平台化：加入神经风格迁移与统一菜单，把单游戏演示整理成平台。

第二部分　系统组成及功能说明

2.1　整体介绍

系统在单块 AXU3EG 板卡上分为三个物理层次：ARM 应用层、命令与数据搬运层、PL 像素后端与显示层。三者通过两类总线耦合——命令走 AXI DMA（8 槽 × 1 MiB 映射缓冲），控制与状态走 AXI-Lite（8 位地址空间），像素走 AXI4 写回 PS DDR 保留区。

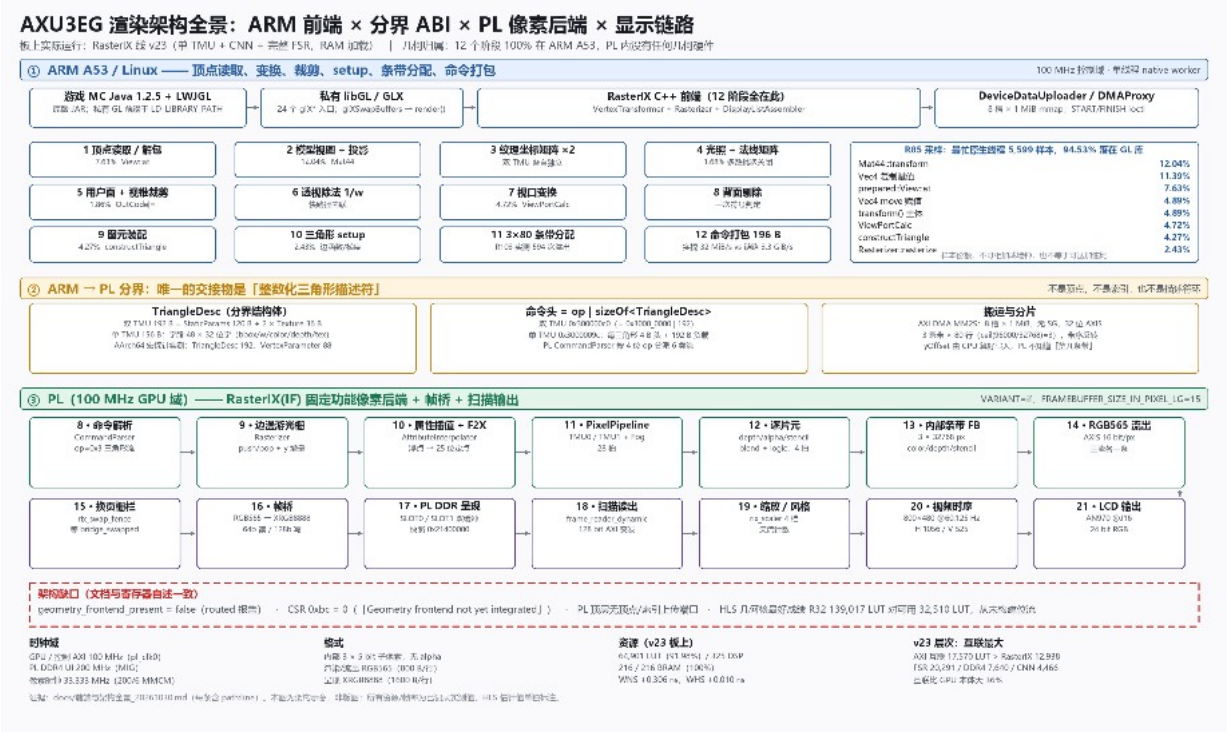


图 2-1　系统架构全景：ARM 十二阶段几何前端、192 字节描述符分界、PL 二十一环节像素后端

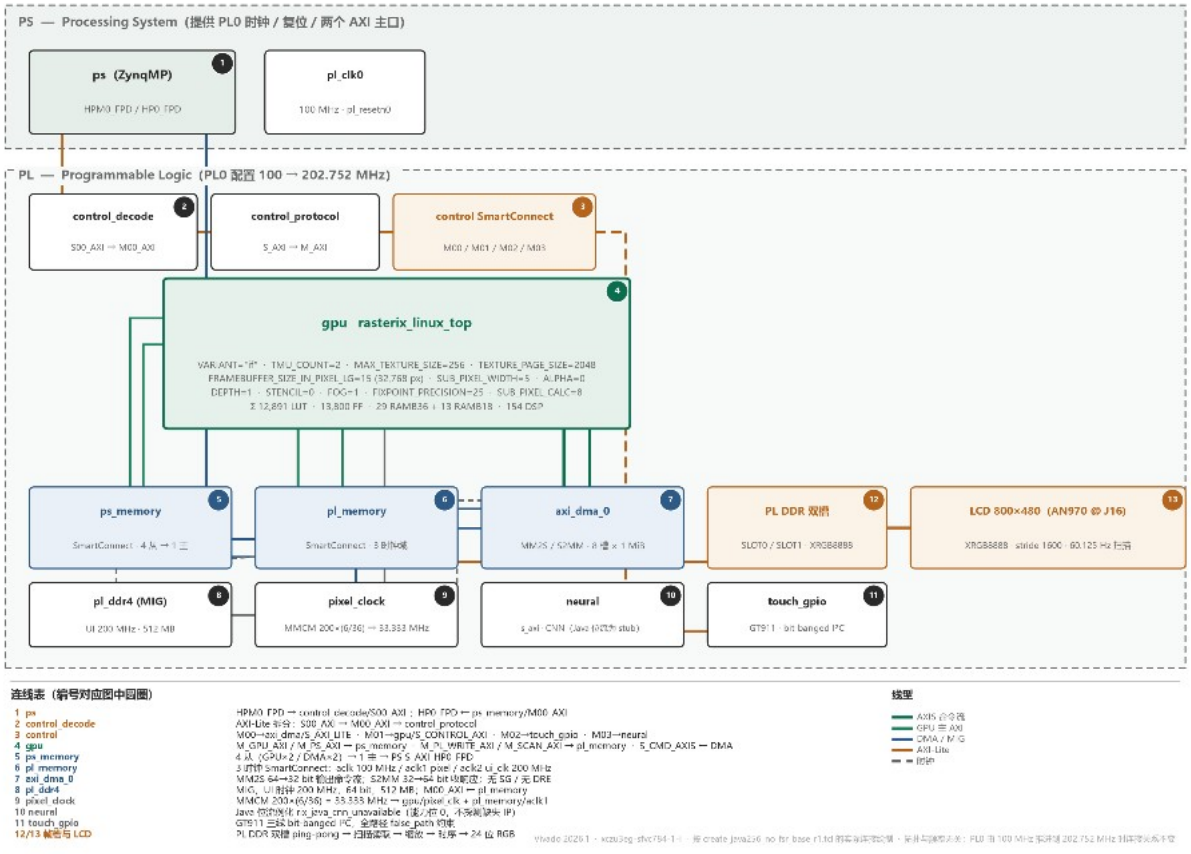


图 2-2 Vivado Block Design 连接拓扑（按工程脚本实际连接绘制，编号对应图下连线表）

图中橙色分界线是本文档的关键：ARM 交给 PL 的最后一样东西不是顶点，而是「已算好边函数与全部梯度的定点三角形参数包」。这一事实决定了 PL 侧不需要重建几何硬件，也决定了后续所有资源讨论的边界。

模块间的依赖关系可概括为三条链：① 命令链：游戏 → 私有 GL 库 → 顶点工作线程 → 显示列表 → DMA → 命令解析器 → 光栅器；② 像素链：光栅器 → 属性插值 → 纹理单元 → 逐片元 → 条带帧缓冲 → RGB565 流出；③ 呈现链：帧桥 → PL DDR 双槽 → 扫描读取 → 缩放 → 时序 → LCD。三条链在「条带帧缓冲出口」与「帧桥入口」两处交汇，交汇点的所有权由四层屏障保护。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

主器件为 XCZU3EG-SFVC784-1-I（Zynq UltraScale+ EG，四核 Cortex-A53 + Mali-400

MP2）。本作品不使用 Mali，其几何前置与像素前端分别由 A53 与可编程逻辑承担，原因在 2.2.3 节展开。板卡提供 2 GB PS DDR4、512 MB PL DDR4、千兆以太网、USB Host、SD 与 eMMC 启动、以及一路 40 针 RGB LCD 接口。

频率方面，本作品已完成从 100 MHz 到 202.752 MHz 的**实测推进**。PL0 由 PS 的 IOPLL 分频产生，配置写 200 MHz，实际分频得到 **202.752 MHz**（周期 4.9321 ns）；签核按 **203.004 MHz**（4.9260 ns）保守约束，整机布线 WNS **+0.224 ns**、WHS **+0.011 ns**，零失败端点。CNN 使用独立的 PL1 时钟域，250 MHz 请求、按 **254.001 MHz** 签核，WNS **+0.116 ns**。超分保持独立的 200 MHz 域（MMCM 200×(6/36)=33.333 MHz 是像素域，两者不同）。中间还实测过 120 / 125 / 133 / 150 / 160 / 166 / 180 / 200 MHz 共八档，**225 MHz 出现负裕量（WNS −0.037 ns）**，因此 202.752 MHz 是当前已验证的上限。

系统存在三个时钟域，必须分别约束：GPU 与控制 AXI 共 100 MHz（来自 PS 的 pl_clk0）；PL DDR4 用户接口 200 MHz（MIG）；像素时钟 33.333 MHz（由 200 MHz 经 MMCM 六分频得到）。像素域与 MIG 用户时钟同源但异步于控制域，因此所有跨域传递都采用显式同步结构而非依赖工具推断。

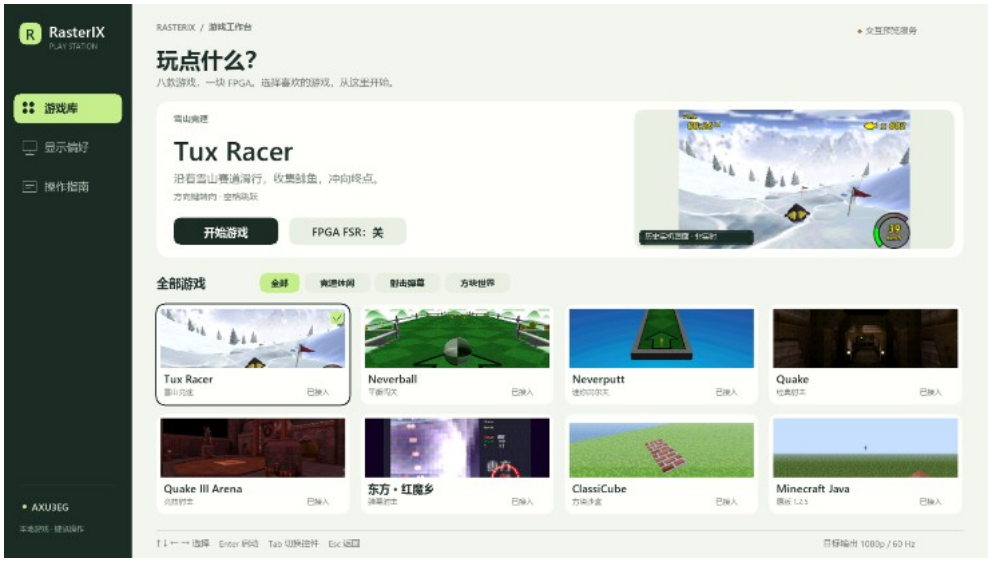


图 2-2 板上运行的图形菜单（1080p 布局，实际输出至 800×480 RGB LCD）

2.2.2 机械设计介绍

本作品为纯电子系统，不涉及机械结构设计。整机由开发板、5 V 电源适配器、USB 键鼠与一根 40 针 FPC 排线（连接 RGB LCD）构成，装在亚克力托板上，无运动部件，因此不提供

CAD 图。若需描述装配关系，可概括为三层：底层为托板与电源，中层为开发板与散热片，顶层为 LCD 模组；三层之间只有 FPC 排线与 USB 线两处柔性连接，便于拆装与运输。

2.2.3 电路各模块介绍

电路层面分三个自制/定制模块与一个既有模块，逐级说明如下。

（1）板级顶层与引脚约束。板级顶层只做三件事：例化系统包装、连出 DDR4 与 LCD 引脚、用一个 ODDR 原语由像素时钟双沿产生 LCD 像素时钟。LCD 采用 24 位并行 RGB、独立 HSYNC/VSYNC/DE，IO 标准为 LVCMOS33，并对输出施加 −2.000/+5.000 ns 的输出延迟约束。关键信号线为：lcd_dclk、lcd_hs、lcd_vs、lcd_de、lcd_rgb[23:0]、lcd_bl_pwm。

（2）GPU 像素后端。这是资源占用最大的模块，内部层次为：命令解析器 → 边漫游光栅器 → 属性插值器与浮点转定点器 → 像素流水线（单 TMU + 雾，Java 双 TMU 配置为双单元）→ 逐片元流水线（颜色混合、逻辑操作、alpha/深度/模板测试）→ 三块内部条带帧缓冲。命令解析器为三态机（等待空闲 / 接收命令 / 执行流），按 4 位操作码 + 28 位立即数分派六类命令流。光栅器不采用完整边函数遍历，而是「找左边界入栈 → 行内右移至边界 → 出栈回起点 → 行增量」的增量算法，输出为「像素 + 控制命令」混合流，向下游插值器发送 X 增量、Y 增量、入栈、出栈四类指令。

（3）帧桥与呈现链。帧桥从 PS DDR 的三个 1 MiB 对齐色槽读取 RGB565，以 16 拍突发为单位缓存后按位复制扩展为 XRGB8888，写入 PL DDR 的两个 1 MiB 乒乓槽，并在需要时同步复制到快照区。呈现控制器用 bundled-data 翻转握手把槽号与帧号送过时钟域，只有在「垂直消隐且扫描器空闲」时才发布新槽，并回送呈现确认。扫描读取器以单笔 AXI4 未完成读按 256 拍边界切分，把 128 位宽的一拍拆成四个像素送缩放器。



图 2-3　三个时钟域、五类跨域同步结构与整机时序收敛结果（v23 routed）

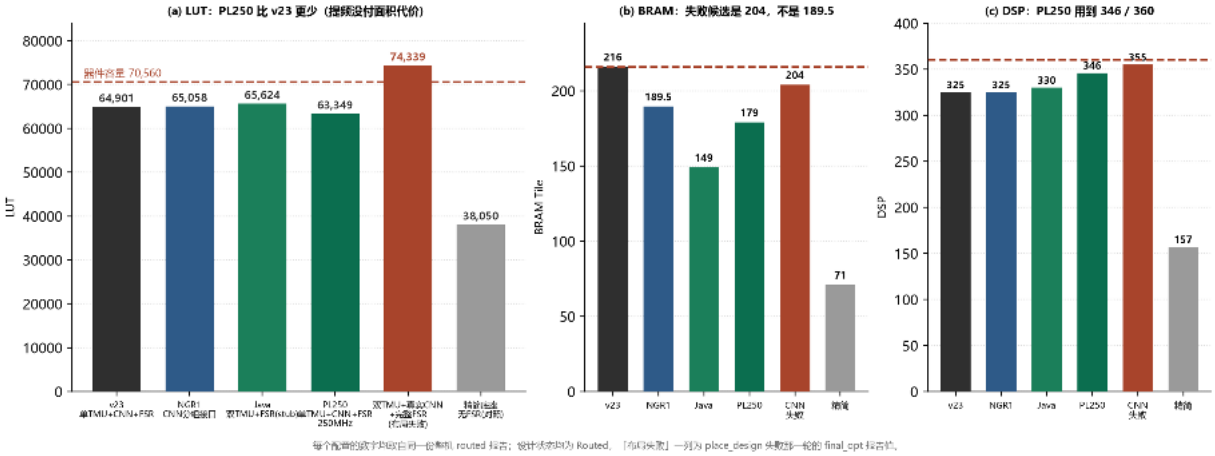


图 2-4　四种位流配置的资源对比：LUT / DSP / BRAM 与器件容量线

（4）关键输入输出信号线。命令侧为 32 位 AXI-Stream（tdata/tvalid/tready/tlast）；状态侧为 8 位地址 / 32 位数据的 AXI-Lite；像素侧为 64 位 GPU 写 AXI 与 128 位扫描读 AXI；DDR 侧为两条独立 AXI4（PS DDR 与 PL DDR）；显示侧为前述 LCD 并行接口。

2.3　软件系统介绍

2.3.1　软件整体介绍

软件分四层：游戏本体、私有 GL 实现、板级适配与驱动、以及主机侧工具链。私有 GL 实现是本作品的核心软件资产——它把游戏的桌面 OpenGL 调用翻译成一条「整数化三角形描述符」命令流，并对上层保持标准 GLX 接口不变，因此可以运行未修改的商业游戏。

为了让读者建立整体印象：该 GL 实现通过一个共享库注册 24 个 GLX 入口，游戏的每一次画面交换最终都收敛到同一个函数，由此触发「显示列表交换 → DMA 提交 → 硬件呈现」三步动作。这条收敛路径使得「一帧」在软件与硬件两侧有唯一对应的定义，也让后面所有帧率与计数都能对齐到同一事件。



图 2-3　八款游戏在开启片内超分后的实际画面回读（RGB565 源图，未做空间缩放）

2.3.2　软件各模块介绍

以下按调用顺序自顶向下给出各模块职责与其关键输入输出变量。

（1）GLX 分派层。职责：向上提供标准 GLX 符号，向下把交换请求转成一次显示列表提交关键输入：glXSwapBuffers 的 display/drawable 句柄；关键输出：一次 render() 调用。该层同时负责在进程启动时校验硬件身份（ABI 与配置哈希），不匹配则拒绝运行。

（2）顶点工作线程。职责：解码命令流、逐顶点变换、图元装配、裁剪、视口、剔除、三角形参数准备、三分片拆分、命令打包。关键输入：88 字节原始顶点与当前矩阵/光照/裁剪状态；关键输出：192 字节整数化三角形描述符。该线程与主线程通过双缓冲显示列表解耦，主线程写、工作线程读，交接点用一次等待保证不覆盖在飞数据。

（3）批量上传线程。职责：把每个条带的显示列表按槽提交给 DMA，并在复用槽之前等待上一次完成。关键输入：条带索引与字节数；关键输出：DMA 启动请求与完成确认。该线程的存在使得「命令生成」与「命令搬运」可以重叠，但也引入了一个必须被正确处理的约束：同一时刻每个通道只允许一笔在途传输。

（4）DMA 连接器与内核模块。职责：把 8 个 1 MiB 槽映射到用户态，提供启动/完成两个 ioctl，并保证复用槽前上一次传输已排空。关键输入：槽号与长度；关键输出：完成状态码。内核侧在超时或错误时会先同步终止通道再归还缓冲，避免用户态在错误路径上复用脏缓冲。

（5）呈现栅栏。职责：把「交换请求」变成「确认呈现」，避免软件以为换了页而硬件仍在写。关键输入：硬件帧号；关键输出：呈现确认。实现上以 300 微秒为轮询步长（内核的 2 毫秒睡眠会被调度粒度放大，实测反而更慢），并对状态做 ABI 与配置双向校验。

（6）游戏菜单与会话管理。职责：承载八款游戏的图形入口、显示偏好、全局返回热键，并按「正常关闭 → 排空 → 恢复位流 → 重建菜单」顺序切换。关键输入：按键与鼠标事件、显示偏好；关键输出：子进程启动与返回请求。菜单本身不走 3D 管线，而是把完整的 800×480 缓冲直接写进 PL DDR 的非活动显示槽，因此小字号文字不会被低分辨率采样破坏——这是我们在实测中发现并修正的一个缺陷。

（7）主机侧工具链。职责：源码获取与固定、参数生成、顶层生成、综合与实现、证据回读与独立验收。关键输入：固定提交号与 SHA 清单；关键输出：可复现的位流、库、启动包与验收报告。该层的设计原则是「任何结论都必须能被另一台机器从固定源重建」。

第三部分　完成情况及性能参数

3.1　整体介绍

系统已在实际硬件上完成端到端运行，不是原理验证或仿真结果。当前板上运行的是「基线像素后端 + 完整超分」这一此前已通过验收的组合，并通过同一套命令入口承载八款游戏；神经网络风格迁移作为独立可选链路保留，已在实机完成固定输入与显示回归，但尚未串入普通游戏菜单，原因是它与超分在同一器件的资源尚未同时满足（详见 3.3 节与 4.1 节）。

一台完整系统包括：AXU3EG 开发板、5 V 电源、40 针 RGB LCD（800×480）、USB 键盘与鼠标、SD 启动卡。上电后由片上启动固件装载 PL 配置并引导 Linux，用户可直接进入图形菜单选择游戏。

3.2　工程成果

3.2.1　机械成果

无机械部件，本节不适用。

3.2.2　电路成果

电路成果为三份可由固定源重建的硬件交付物：基线位流、超分位流、以及超分 +神经风格候选位流。三者均附完整实现报告（资源、时序、DRC、CDC、总线偏斜）与位流摘要，并在板上通过真实加载与图像回读验证。

四个配置的能力与资源**必须逐列对照**，不能跨列引用。下表把「能力」与「资源」分开列：

配置	TMU / 纹理 / 模板	CNN	LUT	FF	BRAM	DSP	WNS / WHS	状态
v23（当前 SD 内位流）	1 / 128 / 无	真实	64,901	71,349	216 / 216	325	+0.306 / +0.010	已上板
NGR1（CNN 分组接口）	1 / 128 / 无	真实	65,058	71,711	189.5 / 216	325	通过	已上板

配置	TMU / 纹理 / 模板	CNN	LUT	FF	BRAM	DSP	WNS / WHS	状态
Java 双 TMU	2 / 256 / 无	stub	65,624	74,028	149 / 216	330	通过	已上板
PL250（上板最高）	1 / 128 / 无	真实	**63,349**	**68,204**	**179 / 216**	**346**	**+0.224 / +0.011**	**已上板**
双 TMU + 真实 CNN + 完整 FSR	2 / 256 / 无	真实	74,339	76,530	204 / 216	355	—	**布局失败**
精简底座（对照基线）	2 / 256 / 无	无	38,050	43,594	71 / 216	157	+1.252 / +0.010	仅布线

三点必须说明。**第一**，本作品**不启用模板缓冲**（`ENABLE_STENCIL_BUFFER=0`），因此依赖模板的渲染效果在游戏侧被关闭；这是已知取舍，不是遗漏。**第二**，上表最后一行的双 TMU + 真实 CNN 组合**布局失败**（105.36% LUT），它的 204 BRAM / 355 DSP 是 `place_design` 失败那一轮的报告值，与 NGR1 的 189.5 BRAM **不是同一个配置**，不可混用。**第三**，PL250 的 63,349 LUT 比 v23 的 64,901 更少，因为改用了稀疏 CNN 检查点；**提频没有付出面积代价。**

需要说明的是，上表第三行的精简底座是本作品为「几何是否可能进 PL」这一问题准备的对照基线：去掉超分与神经网络后，剩余空间为 32,510 LUT、203 DSP。我们在同一基线上尝试过完整的几何硬件化，最好的候选仍需 91,645 LUT，是剩余空间的 2.82 倍，因此该方向被明确否决，而不是「尚未优化完成」。

3.2.3 软件成果

软件成果包括：私有 GL 实现（固定上游提交上的可审查补丁集）、八游戏统一图形菜单、全局返回热键与键盘监视器、显示偏好持久化、以及完整的主机侧工具链。

菜单为中文界面，支持全部/竞速休闲/射击弹幕/方块世界四类筛选、方向键与鼠标双操作、Tab 焦点切换、Esc 返回、弹窗焦点隔离。LCD 版本采用原生 800×480 输出，小字号字形不小于 16 像素，避免经过低分辨率采样后再放大的模糊。

每一款游戏拥有独立的私有库与硬件 profile，切换时会校验位流摘要、库摘要与能力寄存器，建立本次启动的新凭据；任一步失败即退回上一位流，并保留隔离记录供事后分析。

3.3 特性成果

以下逐项给出可量化指标，每项均标注测试条件与口径。

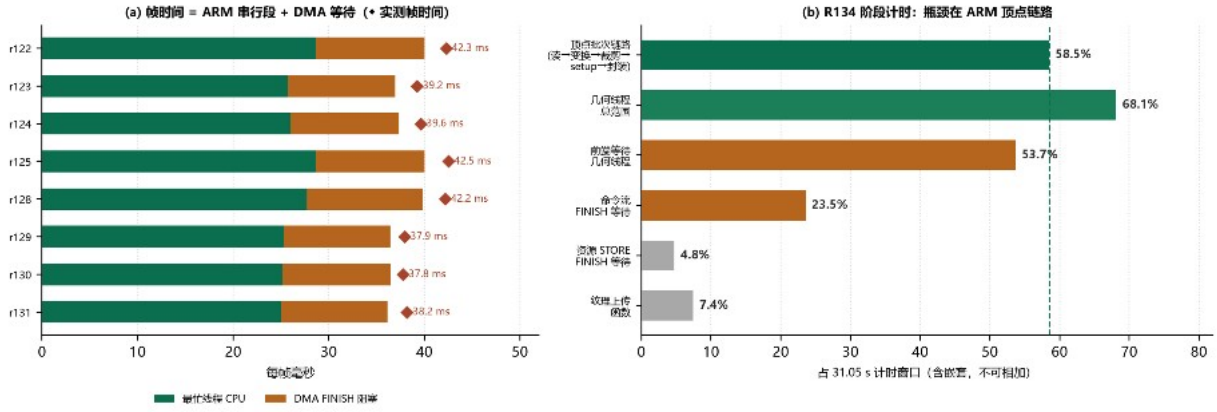


图 3-1 帧时间分解（左：八轮实测；右：R134 阶段计时，瓶颈在 ARM 顶点链路）

（1）频率推进与上板最高配置（PL250）。这是本作品 2026-11-04 的最新进展。GPU 时钟域从 100 MHz 逐步推到 **202.752 MHz**（PS IOPLL 实际分频值，签核按 203.004 MHz 保守约束），整机 WNS **+0.224 ns**、WHS **+0.011 ns**、失败端点 0；CNN 走独立的 PL1 域，250 MHz 请求、按 **254.001 MHz** 签核，WNS **+0.116 ns**。中间实测过 120 / 125 / 133 / 150 / 160 / 166 / 180 / 200 MHz 八档，**225 MHz 出现 WNS −0.037 ns 的负裕量**，因此 202.752 MHz 是已验证上限而不是估计。值得强调：PL250 的 **63,349 LUT 比 v23 的 64,901 更少**，因为改用了稀疏 CNN 检查点，**提频没有付出面积代价。**

在该配置上，十五款游戏全部完成「CNN + PL FSR」串联复测，输出帧率区间 **45.0–55.8 帧/秒**：arcade-breakout 55.81、arcade-snake 55.76、arcade-2048 55.72、arcade-sokoban 55.73、neverputt 55.58、arcade-mines 55.46、arcade-blocks 55.35、classicube 54.72、quake 54.47、rrootage 54.23、quake3 53.95、touhou 52.66、neverball 46.83、tuxracer 45.03、minecraft-java 18.06。CNN 核心稳定在 **5.723 ms**，超分等待在 13 款游戏上仅 **0.015 –0.023 ms/帧**。其中 arcade-snake 另做了 **120 秒连续长测：6,692 帧 / 55.76 帧/秒**，超分等待 0.002 ms/帧，失败计数全为零。

（2）超分长测吞吐。在 64^3 世界、视距 32、关闭风格迁移的条件下，120 秒长测的新输出帧率为 57.089 帧/秒（源 57.107）；缩短视距到 16 后为 60.106 帧/秒（源 60.132）。

需要明确：这一数字是「新输出帧率」，不等于 LCD 每秒显示 60 个不同的新画面——LCD 扫描节拍固定为约 60.125 Hz，当新帧率低于它时显示端会重复扫描最近一帧。

（3）同场景调度对照。在完全相同的位流内，四轮 ABBA 对照显示「提前请求」调度比「普通」调度提高 33.01%；在相同预取调度下，当前版本比前一版本提高约 7.80%，比更早版本提高约 29.87%。20 秒短窗的绝对值为 36.83 → 44.37 → 47.83。版本间是顺序测量，不能把各阶段节省的时间简单相加来预测帧率。

（4）游戏库验收。八款游戏逐一完成「开启超分 → 启动 → 出帧 → 正常退出 → 位流恢复」检查，全部通过，位流与库摘要前后一致，无残留客户端。实测超分处理帧率为：ClassiCube 58.88 / 60.08（两种运行方式）、Tux Racer 60.12、Neverputt 59.97、Quake 53.70、东方 50.66 / 49.12、Quake III Arena 27.87、Neverball 24.84、Minecraft Java 19.93。这些是「超分处理帧率」，与各游戏自身的完成帧率不是同一指标。

（5）CNN 与超分的串联链路。**这里有一个必须精确表述的事实**：CNN 与分级 FSR 之所以能同时存在，不是因为在运行时「分时」而省下了资源——只要两者同时例化，静态 LUT / DSP / BRAM 就必然占用，与是否分时运行**无关**。真实情况是：NGR1 位流本身已经同时包含 CNN 与 FSR（65,058 LUT / 189.5 BRAM / 325 DSP），软件串联**只是复用了这个已有位流**，接入过程没有新增任何 RTL 或位流资源。

而「双 TMU + 真实 CNN + 完整 FSR」这个组合确实装不下：相加需 74,339 LUT，超出 70,560 容量，`place_design` 直接失败（105.36%）。要让双 TMU 与真实 CNN 共存，必须**真正共享运算器或缓存**，或削减配置，不能靠软件分时来论证。

串联后的实测表现：Neverputt 同一球洞的呈现吞吐由 46.68 提升到 50.70 FPS（+8.62%，ABCCBA 八轮、3,882 帧），漫画与油画两套权重分别为 50.71 / 50.63 FPS，显示等待由 3.39 降至 1.69 ms/帧。另有**跨帧并行**版本——上一帧 FSR 与下一帧 CNN 重叠执行——把同一指标从 38.06 提到 46.89 FPS（+23.19%）；其首版因约 29.89 FPS 被拒绝，失败记录保留。**因此「任一时点只有一个模块工作」这个说法在当前实现里并不成立**，跨帧重叠已经落地。

（6）多游戏分别调优。不是所有游戏都适合同一套呈现策略。逐游戏实测后的取舍是：

Neverball 改用异步呈现，实际开局源呈现由 28.97 提升到 45.17 FPS（+55.92%）；Quake III 把内部上限从 30 调到 60，由 30.27 提升到 54.73 FPS（+80.81%）；而 Tux Racer 强制同步反而由约 60 降到 48 FPS，因此保留原策略。CNN 侧的提交时机也逐游戏选择（Neverputt / Quake / Quake III / ClassiCube 用 R3，Tux Racer / Neverball / 东方保留 R2）。**没有降低分辨率、纹理、光照或视距。**

（7）游戏库扩容与自研小游戏。游戏库由八款扩展到十五款，新增 rRootage（上游完整开源游戏）与六款自研原生规则小游戏：贪吃蛇、落块消行、打砖块、2048、推箱子、扫雷。六款规则集中在一个头文件（`ports/arcade/rules.hpp`），绘制统一走固定功能 GL 管线，**不使用 CPU 软件帧缓冲冒充 FPGA 光栅化。**七款新增游戏均通过实机 PL FSR 测试，短测源呈现 40–60 FPS。菜单同时支持分页、搜索、收藏与最近游玩。

（8）神经网络存储回收。CNN 核心 BRAM 由 55 块降至 28.5 块，整机由 216 块降至 189.5 块，DSP 保持 325 不变。**在原子接口、100 MHz 条件下**，推理中位耗时由 12.405 ms 变为 12.873 ms，FPGA 计算等待由 4.148 ms 变为 4.628 ms，核心周期由 414,738 变为 462,816——即用约 11.6% 的延迟增长换取 26.5 块 BRAM 的空间。**但这不是当前最快路径**：改用分组（group）输出接口后为 9.321 ms（−27.6%）；再叠加 250 MHz CNN 时钟后，1,430,784 周期 = **5.723 ms。**三个数字对应「原子/100 MHz」「分组/100 MHz」「分组/250 MHz」三种运行时，必须分别标注。

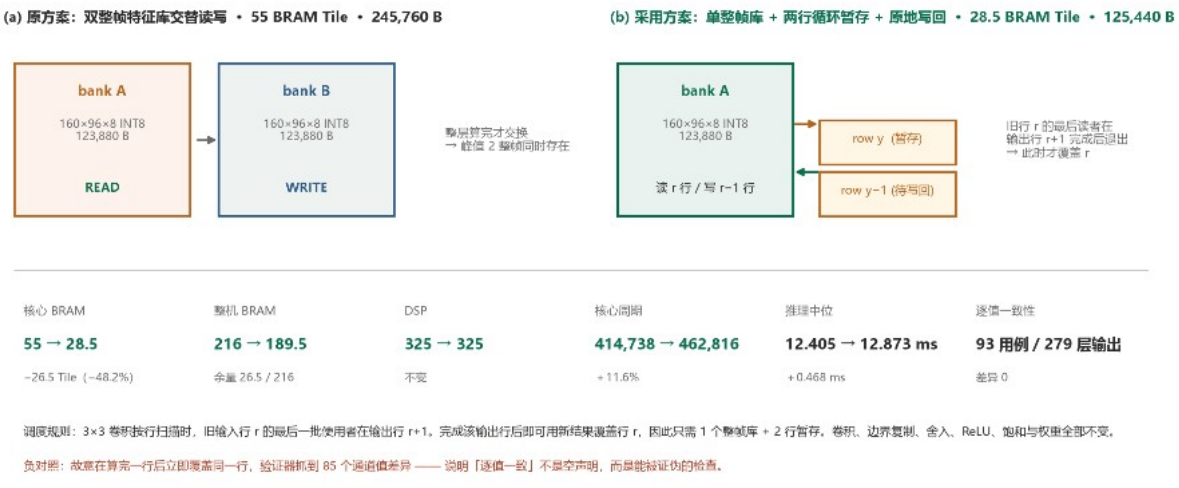


图 3-2 CNN 特征库调度：双整帧库 → 单整帧 + 两行暂存，回收 26.5 块 BRAM

（9）图像一致性——**证据的边界必须说清楚。**超分链路在板上完成整帧 384,000 像素的逐

像素比对。CNN 链路的 36 帧、共 4,423,680 字节，折合每帧 122,880 字节= 160×96×**8 通道** INT8，这是 **HWC8 中间张量，不是最终三通道 RGB 输出。**因此它证明的是「PL 卷积结果与 CPU INT8 参考逐字节一致」，**不能单独证明 ARM 融合与 FSR 之后的最终画面正确。**

最终画面正确性由三条独立证据支撑：融合之后的整帧比对、FSR 输出的整帧 384,000 像素比对、以及每场景 6,000 点 LCD 抽样。PL250 配置下另做过 15,360 像素的 CPU vs PL 逐字节检验（`PL250_CNN_EXACT 15360`，SHA256 `cb1c5edb…`）。这些检查共同证明「实现符合定义」但**不自动证明画质主观更好**——画质评价以同源对照图另行给出。

（10）帧所有权保护。四层屏障在实机上接受了 1,000 次连续换页、延迟写响应、写响应非零、扫描欠流、DMA 超时等负向注入，任一层异常都会进入故障态并拒绝继续提交，而不是以「画面撕裂但计数正常」的形式静默通过。

第四部分　总结

4.1　可扩展之处

本作品有三处已被证实、但尚未合并进默认版本的方向。

第一，超分与神经网络的同时启用。当前两者各自可运行，但同时装入同一器件需要 74,339 LUT，超出 70,560 的容量约 3,779 个，布局阶段即失败。已确认的可行方向是裁剪超分的锐化级、或进一步复用两者的行缓存；这属于设计取舍，需要按目标画质取向决定，而不是单纯优化问题。

第二，几何硬件的局部化。完整硬件化已被否决，但「只硬化无裁剪、无光照、单位纹理矩阵的地形批次」仍有可能落进剩余预算。前提是先把每帧的顶点与三角形数量分布测准，目前该数据尚不完整。

第三，外接高分辨率显示。板卡具备 DisplayPort 通道，当前状态为未连接；要输出 1080p60 需要建立独立显示进程、完整帧发布协议与 DRM/KMS 缓冲管理，且必须与现有 LCD

通路并存而非替换。

4.2 心得体会

这个项目从「让一个商业游戏在一块小器件上动起来」开始，最后变成了一次关于「如何在约束里做取舍」的长期训练。回顾整个过程，我认为最值得记录的不是那些成功的数字，而是几次判断失误以及它们被纠正的方式。

第一次误判发生在架构层面。我们最初相信「只要把几何搬进 FPGA，帧率就会上去」，于是投入了大量精力做完整几何核的硬件化。面积从 374,209 LUT 一路压到 91,645 LUT，减少了一半以上，这本身是扎实的工程成果；但可用空间只有 32,510 LUT，差距是 2.82 倍而不是 2.82%。这件事教会我：优化的相对幅度和目标的绝对差距是两件事，前者可以很漂亮，后者可能毫无变化。后来我们把这条结论写成文档里的一句硬话——「再重构三轮也装不下」，并要求后续任何相关讨论都必须先引用它，避免重复劳动。

第三次误判是最近发生的，也最让我警醒。我基于一份独立模块的综合报告推算「加上新功能后仍然装得下」，并把这个判断当作计划的前提。直到真正做整机实现时才看到，那个模块单独综合的 4,466 LUT 在整机里变成了 8,265 LUT —— 因为新增的数据通路会让相邻模块一起长大。孤立模块的数字不能直接相加，这是硬件工程的常识，但当我急着推进时还是犯了。现在我们的做法是：任何资源结论只认整机实现报告，模块级报告只能用于判断「哪个模块值得改」。

与之相对，有几件事我认为做对了。一是把「失败」也当作可交付物：被否决的候选、被撤回的结论、甚至验证器本身的 bug，都保留在文档里并注明原因，让后来的讨论能从更靠前的位置开始。二是坚持「一个数字一个口径」：源帧率、增强完成帧率、LCD 扫描频率严格分开记。三是把「可复现」当作功能：每份位流与报告都能指回固定的上游提交。

在方法层面，我最大的收获是学会了「用测量代替推测」。项目中期我们做过一次纯软件优化，采样显示矩阵变换与向量拷贝占了工作线程三成以上时间，于是很自然地认为「把这部分搬进硬件会有同样比例的收益」。但真正做了阶段计时之后才发现，瓶颈确实在这一段，可它是与其他环节串行耦合的，单独加速一段并不能线性换算成帧率。这类「看起来显然」的推断，是这类项目里最容易出错的地方。

最后是工程习惯。这个项目有过因为串口被占用而让板上试验卡住的经历，也有过因为一次不安全终止而触发隔离机制、必须整板重启的经历。这些琐碎的约束后来都变成了流程：脚本会检查隔离文件、会在切位流前先构造反向回滚包、会在每一步之后回读核对摘要。在一个「一次操作就可能把现场弄坏」的系统上，把恢复能力设计好，比把功能做多更重要。

第五部分　参考文献

[1] AMD. Zynq UltraScale+ Device Technical Reference Manual (UG1085). Advanced Micro Devices, 2024.

[2] AMD. UltraScale Architecture Memory Resources (UG573). Advanced Micro Devices, 2024.

[3] AMD. Vitis High-Level Synthesis User Guide (UG1399). Advanced Micro Devices, 2024.

[4] AMD. Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906). 2024.

[5] AMD. AXI DMA v7.1 LogiCORE IP Product Guide (PG021). Advanced Micro Devices, 2023.

[6] ARM. AMBA AXI and ACE Protocol Specification, ARM IHI 0022F. ARM Limited, 2020.

[7] ARM. Cortex-A53 Processor Technical Reference Manual (ARM DDI 0500). ARM Limited, 2018.

[8] ARM. Mali-400 MP GPU Developer Guide. ARM Limited, 2019.

[9] Xilinx. RasterIX: Fixed-Function OpenGL Rasterizer for FPGA, commit 9269a01. https://github.com/Xilinx/RasterIX

[10] AMD GPUOpen. FidelityFX Super Resolution 1 (FSR1): EASU and RCAS, commit a21ffb8f. https://github.com/GPUOpen-Effects/FidelityFX-FSR

[11] AMD GPUOpen. FidelityFX Super Resolution Algorithm Documentation. 2021.

[12] Khronos Group. OpenGL 1.3 / GLX 1.3 Specification. Khronos, 2001.

[13] Khronos Group. OpenGL ES 2.0 Specification. Khronos, 2004.

[14] Segal M, Akeley K. The OpenGL Graphics System: A Specification (Version 1.4). 2004.

[15] Pineda J. A Parallel Algorithm for Polygon Rasterization. SIGGRAPH 1988, pp. 17-20.

[16] Akenine-Möller T, Haines E, Hoffman N. Real-Time Rendering, 4th ed. CRC Press, 2018.

[17] Fuchs H, Poulton J, et al. A Shading and Rendering Architecture for the Sort-Middle Class of Graphics Systems. IEEE TCAD, 1989.

[18] Jacob B, Ng S, Wang D. Memory Systems: Cache, DRAM, Disk. Morgan Kaufmann, 2008.

[19] Cumming D. FPGA Design: Best Practices for Team-based Reuse. Springer, 2015.

[20] Linux Kernel Documentation. DRM/KMS Kernel Modesetting and Framebuffer API. 2024.